==========

APEX RETAIL GROUP | ANALYTICS TRAINING PROGRAM
DOCUMENT: CLASS NOTES (SESSION 3)
TOPIC: ARRAY OPERATIONS & METRIC CALCULATION — VECTORIZED ANALYSIS
LEVEL: BEGINNER (DAY 3)

==========

1. CONTINUITY & BUSINESS SCENARIO RECAP

In Session 1, you calculated individual salary variances using manual list index positions.
In Session 2, you automated rule checking using `for` loops, functions, and conditional statements.

Now, in your third week at Apex Retail Group:

Elena Rostova, Regional Sales Director, reaches out with an urgent operational challenge:

> "**Our regional sales data is logged across four major store clusters over four quarters. We need to evaluate quarter-over-quarter sales targets, calculate regional performance variances, and measure sales dispersion (variance and standard deviation) across all clusters.**
>
> **Iterating through millions of numbers using Python `for` loops is far too slow and cumbersome for enterprise reporting. We need a mathematical tool designed for multi-dimensional numerical operations**.
>
> **Can you take these quarterly store cluster arrays, calculate vectorized growth and dispersion metrics, and flag which regional clusters missed target baselines?"**

In this session, you step beyond pure Python lists and introduce NumPy (Numerical Python).
You will learn how to perform vectorized math across arrays, inspect matrix dimensions,
and apply statistical functions without writing loops.

2. WHY NUMPY? THE NEED FOR VECTORIZED OPERATIONS

A standard Python list is a flexible sequence of pointer objects. Performing mathematical operations on every element in a list requires explicitly writing a `for` loop:

```
# Python List Approach (Requires Loop)
sales = [120000.0, 150000.0, 98000.0]
taxed_sales = []
for s in sales:
    taxed_sales.append(s * 1.10)
```

NumPy introduces the N-dimensional array (`ndarray`). In an array, data is stored in contiguous memory blocks. This enables VECTORIZATION—applying a single operation to an entire array simultaneously without writing `for` loops:

```
# NumPy Vectorized Approach (No Loop Required)
import numpy as np
sales = np.array([120000.0, 150000.0, 98000.0])
taxed_sales = sales * 1.10  # Multiplies all elements instantly!
```

3. NUMPY ARRAY CONSTRUCTS & MULTI-DIMENSIONAL SHAPES

A. Array Dimensions and `.shape`
- 1D Array (Vector): A single row or column of numbers. `shape` is `(N,)`.
- 2D Array (Matrix): Rows and columns of numbers (e.g., Stores x Quarters). `shape` is `(Rows, Columns)`.

Example:

```
import numpy as np

# 2D Array representing 4 Store Clusters across 4 Quarters
sales_matrix = np.array([
    [150000, 162000, 158000, 171000],  # Cluster 1
    [210000, 205000, 215000, 228000],  # Cluster 2
    [95000,  89000,  92000,  98000],   # Cluster 3
    [180000, 195000, 188000, 202000]   # Cluster 4
])
print(sales_matrix.shape)  # Output: (4, 4)
```

B. Slicing Multi-Dimensional Arrays

Array slicing uses `[row_slice, column_slice]`:

- Select Cluster 1 (Row 0, All Columns): `sales_matrix[0, :]`
- Select Q1 Sales across all Clusters (All Rows, Column 0): `sales_matrix[:, 0]`

C. Vectorized Arithmetic & Boolean Masking

- Subtracting target benchmarks: `variance = sales_matrix - target_matrix`
- Boolean filtering (Masking): `missing_targets = sales_matrix < 100000`

4. STATISTICAL AGGREGATIONS & AXIS CONTROL

NumPy provides high-performance statistical aggregations. When working with 2D matrices,the `axis` parameter specifies the direction of calculation:

- `axis=0`: Calculates vertically DOWN columns (e.g., total sales per quarter across all clusters).
- `axis=1`: Calculates horizontally ACROSS rows (e.g., total sales per cluster across all quarters).

Function	Business Meaning
`np.mean(sales, axis=1)`	Average quarterly revenue per cluster
`np.sum(sales, axis=0)`	Total enterprise revenue per quarter
`np.var(sales)` & `np.std(sales)`	Spread / volatility across store clusters

5. EXECUTING THE REGIONAL SALES ANALYSIS IN NUMPY

Below is the clean, annotated Python code required for Elena's regional audit:

```python

import numpy as np

# Step 1: Define 2D Matrix of Quarterly Sales (4 Clusters x 4 Quarters) in USD
# Rows = Clusters (North, South, East, West) | Cols = Quarters (Q1, Q2, Q3, Q4)
sales_data = np.array([
    [150000.0, 162000.0, 158000.0, 171000.0],  # North Cluster
    [210000.0, 205000.0, 215000.0, 228000.0],  # South Cluster
    [95000.0,  89000.0,  92000.0,  98000.0],   # East Cluster
    [180000.0, 195000.0, 188000.0, 202000.0]   # West Cluster
])

# Step 2: Define Quarterly Target Matrix ($160,000 baseline per quarter per cluster)
quarterly_target = 160000.0

# Step 3: Vectorized Calculations (No loops required!)
```

```
cluster_totals = np.sum(sales_data, axis=1)         # Sum across columns
(axis=1)
cluster_averages = np.mean(sales_data, axis=1)       # Mean across columns
(axis=1)
quarterly_totals = np.sum(sales_data, axis=0)       # Sum down rows (axis=0)

# Vectorized Variance against Target
variance_from_target = sales_data - quarterly_target

# Dispersion / Volatility metrics across entire enterprise
overall_mean = np.mean(sales_data)
overall_std = np.std(sales_data)

# Boolean Masking to isolate underperforming quarters (< $160k target)
underperforming_mask = sales_data < quarterly_target

print("--- REGIONAL SALES AUDIT RESULTS ---")
print("Cluster Total Revenue ($):", cluster_totals)
print("Cluster Average Revenue ($):", cluster_averages)
print("Quarterly Enterprise Totals ($):", quarterly_totals)
print("Overall Revenue Std Deviation ($):", round(overall_std, 2))
print("Underperforming Quarters Mask:\n", underperforming_mask)
```

INTERPRETING NUMBERS & FORMULATING BUSINESS RECOMMENDATIONS

A. Observation (The Raw Data):

- Total Enterprise Revenue: $2,538,000.00 across 4 clusters and 4 quarters.
- Cluster Performance:
  - South Cluster: Highest total revenue ($858,000.00), average $214,500/quarter.
  - West Cluster: Second highest ($765,000.00), average $191,250/quarter.
  - North Cluster: Third highest ($641,000.00), average $160,250/quarter.
  - East Cluster: Severe lag ($374,000.00), average $93,500/quarter.
- Dispersion: High overall standard deviation ($45,713.88), indicating severe performance inequality between regional clusters.
- Target Breaches: East Cluster missed the $160,000 target in ALL 4 quarters. North Cluster missed target in Q1 ($150k) and Q3 ($158k).

B. Interpretation & Business Meaning:

- While the overall enterprise appears healthy, performance is heavily skewed by the South and West clusters.
- The East Cluster is structurally underperforming, averaging 41.5% below target ($93.5k vs $160k target).
- High sales dispersion ($45.7k std dev) proves that regional execution is inconsistent across the retail footprint.

C. Professional Recommendation to Elena Rostova (Sales Director):

"Enterprise revenue totals $2.538M, but sales dispersion is critical (standard deviation of $45.7k). The East Cluster failed to hit the $160k target in any quarter, generating only $374k total compared to South's $858k.

Recommendation: Avoid treating regional stores as homogeneous. Reallocate marketing and supply chain budget from the mature South Cluster to restructure store operations in the underperforming East Cluster."

CORE MINDSET HABITS REINFORCED TODAY

1. Measure dispersion, not just averages: An average quarterly cluster revenue of $158,625 hides the reality that East generates $93.5k while South generates $214.5k.
2. Leverage vectorization for scale: Eliminate manual loops when performing uniform mathematical operations across multi-dimensional business grids.